

```
set raw(syntaxes: "files/syntax/cds.sublime-syntax")
```



SAP BTP Pro-Code

Cloud Application Programming (CAP)

Version: 1.3

Date: January 14, 2026

Table of Contents

1. Setup	5
2. Task Description	6
2.1. Helpful Links	6
3. Data Model	7
4. Implementation	8
4.1. Initialize CAP Application	8
4.2. Project Structure	8
4.3. File Structure Setup	8
4.4. Create Data Models	8
4.5. Create Example Data	9
4.6. Test Data Models	9
4.7. Create Services	9
4.8. Add Actions	9
4.9. Create UI Applications	9
4.10. Define UI Annotations	11
4.11. Add Local Launchpad	11
4.12. Add Actions to UI	12
4.13. Debug Your Application	13
4.14. Adding Authorization and Authentication	13
4.15. Adding Business Logic	13
5. Optional: Advanced UI Customization	15
5.1. Example UI Screenshots	15
5.2. Refresh UI with SideEffects	16
5.3. Multiple Tabs Configuration	16
5.4. Row Highlighting with Criticality	16
5.5. Conditional Action Buttons	17
6. Optional: Deployment to Cloud Foundry	18
6.1. Add Production Configuration	18
6.2. Fix ui5-deploy.yaml Configuration	18
6.3. Cloud Foundry Login	19
6.4. Build and Deploy	19
6.5. Access Deployed Apps	19
6.6. Cleanup / Undeploy	20

Note: This workshop includes two optional advanced sections at the end:

- **Optional: Advanced UI Customization** – Enhance your UI with SideEffects, criticality highlighting, and conditional buttons
- **Optional: Deployment to Cloud Foundry** – Deploy your application to SAP BTP Cloud Foundry

You may explore these sections based on your time and interest after completing the main exercise.

1. Setup

1. Follow the [Getting Started Page](#) of the Capire Doc
2. Install the following extensions in VS Code:
 - [Cloud Foundry Tools](#)
 - [SAP Fiori Tools - Extension Pack](#)
 - [SAP CDS Language Support](#)
 - [Rainbow CSV](#)

You may need to restart VS Code to load all new extensions.

2. Task Description

Develop a comprehensive full-stack university management application using SAP Cloud Application Programming Model (CAP) that serves as a centralized system for managing academic operations. The application will handle the organization of academic programs (studies), course modules, and student enrollment in a structured and efficient manner.

This enterprise-grade solution will provide:

- **Study Program Management:** Organize and maintain different study programs (degrees) with their associated requirements including credits needed for completion, program duration, and curriculum structure
- **Module Catalog Management:** Define and manage course modules offered across study programs, including credit values, weights, validity periods, mandatory/elective status, exam options, and assignment tracking
- **Student Management:** Track students throughout their academic journey, managing their enrollment in study programs and registration for specific modules, maintaining comprehensive student records including personal information and academic progress
- **Module Assignment Tracking:** Enable students to register for available modules and manage their course selections, while enforcing business rules around module availability, prerequisites, and study program requirements
- **No Worries:** We won't implement all of this ;)

The system establishes clear relationships between Students, Studies (degree programs), and Modules, allowing for flexible curriculum design while maintaining data integrity and providing students with a personalized view of their academic path.

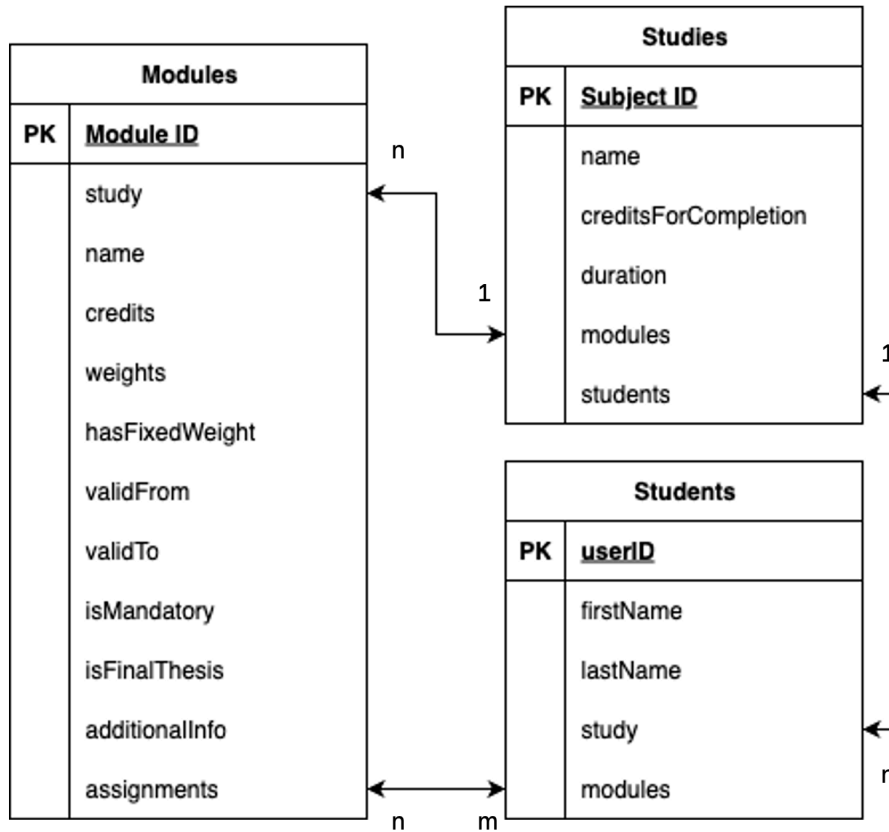
2.1. Helpful Links

While working on the task, feel free to use the following documentations as much as possible. Especially the CAP Documentation is really powerful.

- [Internal CAPire \(CAP Documentation\)](#)
- [Fiori development portal & Fiori elements documentation](#)
- [SAP Learning](#)

3. Data Model

Use below diagram as a reference to design your CAP entities.



4. Implementation

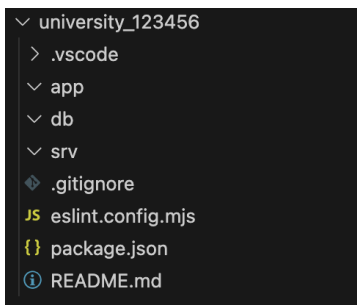
4.1. Initialize CAP Application

Initialize your CAP application with the following command in the terminal either in VS Code or BAS:

```
cds init university
cd university
```

4.2. Project Structure

Your project structure should look like this:



4.3. File Structure Setup

Create the database schema file in /db/:

- schema.cds

Create service definition files in /srv/:

- students-service.cds
- studies-service.cds
- catalog-service.cds

4.4. Create Data Models

Create the data models in the /db folder based on the entity diagram. You can have a look at the [CDL documentation](#) for reference.

Hint: If you are stuck with the creation of the data model, please raise your hand. We are happy to help you.

4.5. Create Example Data

Generate schema with example data:

```
cds add data --records 10
```

4.6. Test Data Models

Test if your data models are defined correctly and no compilation errors occur (You can leave this command running while working on the next steps):

```
cds watch
```

Only proceed to the next step if your compilation is successful, otherwise have a closer look at your data model. In case you make changes to your model, you also may need to delete the db/data folder and recreate the example data.

4.7. Create Services

Create the services in the /srv/ folder, which expose the entities via API. Check out the [documentation](#) for how to define services.

- The studies service should expose the Studies entity
- The students service should expose the Students entity and the Studies entity. The studies entity can be later used as a value help when creating a new student and assigning them to a study
- In the Students and Studies service enable draft for the respective entities to allow them being edited
- In the Catalog service expose the Modules entity as well as the ModuleAssignments entity, to show a list of modules students can sign up for

4.8. Add Actions

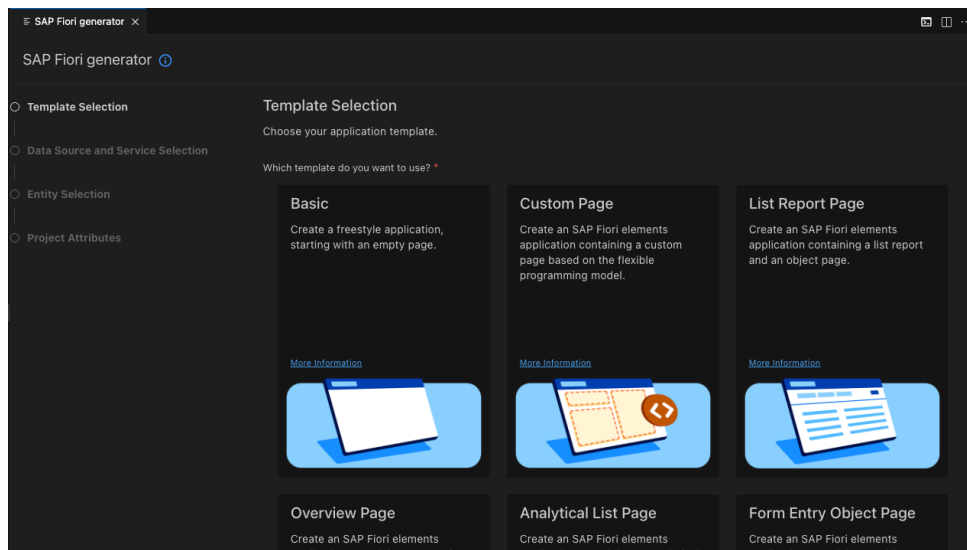
Actions allow to define POST endpoints, which execute some logic when being called. Follow the [docs](#) to add a bound action “assign” to the Modules. Do not add an implementation for that yet. We will do that later.

4.9. Create UI Applications

Create UI applications by using the Fiori Frontend Generator.

Note: You should repeat this step for each OData service, in total 3 times.

1. In the menu click **View** → **Command Palette** (Ctrl + Shift + P) and start typing “Fiori” and select **Fiori: Open CF Application Generator**
2. Select **List Report Page** in the template section and click “Next”



1. In **Data Source** select “Use a Local CAP Project”:
 - Choose a CAP Project: university
 - OData Service: YourService
2. In the **Entity Selection** select:
 - Main Entity: YourMainEntityOfTheService
 - Navigation Entity: YourNavigationEntity (If applicable)
 - Automatically add table columns to the list page and a section to the object page if none already exists: **Yes**
 - Table Type: **Responsive**
3. In the **Project Attributes** select:
 - Module Name: e.g. students-management for the students-service
 - Application Title: e.g. “Students Management” for the students service
 - Application Namespace: empty
 - Description: YourDescription
 - Enable TypeScript: **No**
 - Add Fiori Launchpad Configuration: **Yes**
4. In the **SAP Fiori Launchpad Configuration**:
 - Semantic Object: YourMainEntityOfTheService
 - Action: manage (or something better fitting)
 - Title: Choose accordingly, will be displayed on the Launchpad tile
 - Subtitle: Choose accordingly, will be displayed on the Launchpad tile
5. Continue creating the Fiori config for the remaining services, once you finished, continue with the next step.

4.10. Define UI Annotations

For defining the UI annotations, you can use the Fiori application modeler plugin (



icon):

- Select the app you want to work on and select the page, e.g. either the list page or the details page
- Afterwards you can configure columns for tables or modify the sections shown on the details page
- **FYI, but not necessary:** You can also manually define the annotations in CDS files. Ideally use the `annotations.cds` file in each `app/` folder to do the annotations for the respective app:
 - `@UI.LineItem` specifies a table definition
 - `@UI.HeaderInfo` specifies the header of an object page as well as the table name
 - `@UI.PresentationVariant` specifies the default sorting for a table
 - `@UI.FieldGroup` specifies a form
 - `@UI.Facets` specifies the sections of an object page. Facets can contain field groups, line items or charts
 - You can check-out the [Fiori elements showcase](#) for samples of each annotation
- To disable the “Go” button in the UI add the “liveMode” setting in each applications `webapp/manifest.json` file. For more information, reference to the [docs](#)

4.11. Add Local Launchpad

Add a local launchpad by running the following command:

```
npm add -D cds-launchpad-plugin
```

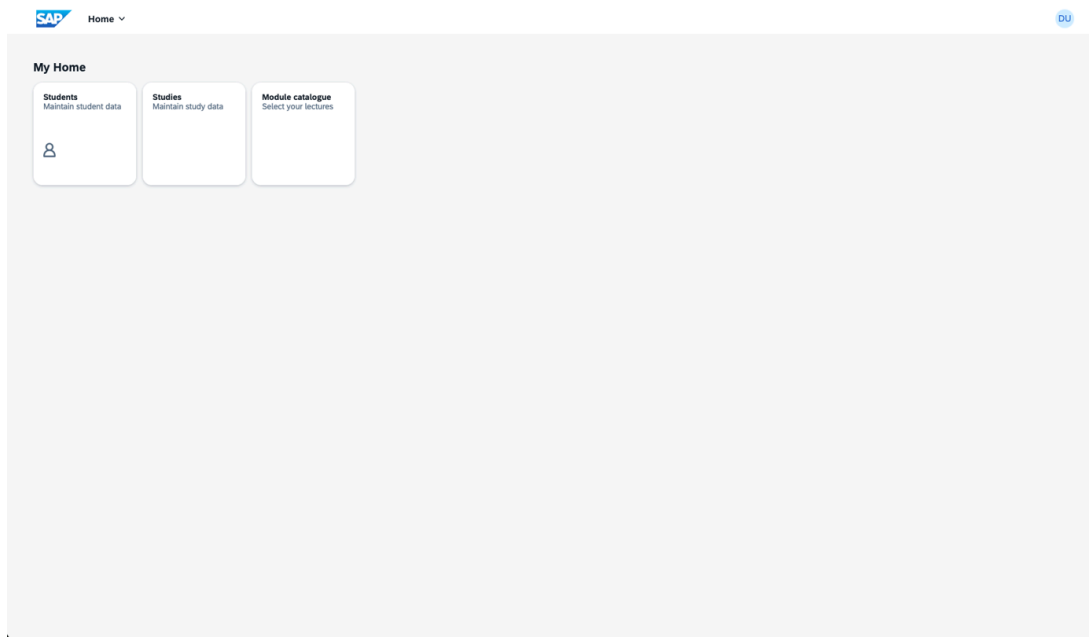
Afterwards usually on localhost, port 4004 a “Sandbox Launchpad” appears, via which you can test the UI apps within the SAP Shell.

Welcome to @sap/cds Server

Serving tum-sample 1.0.0

These are the paths currently served:

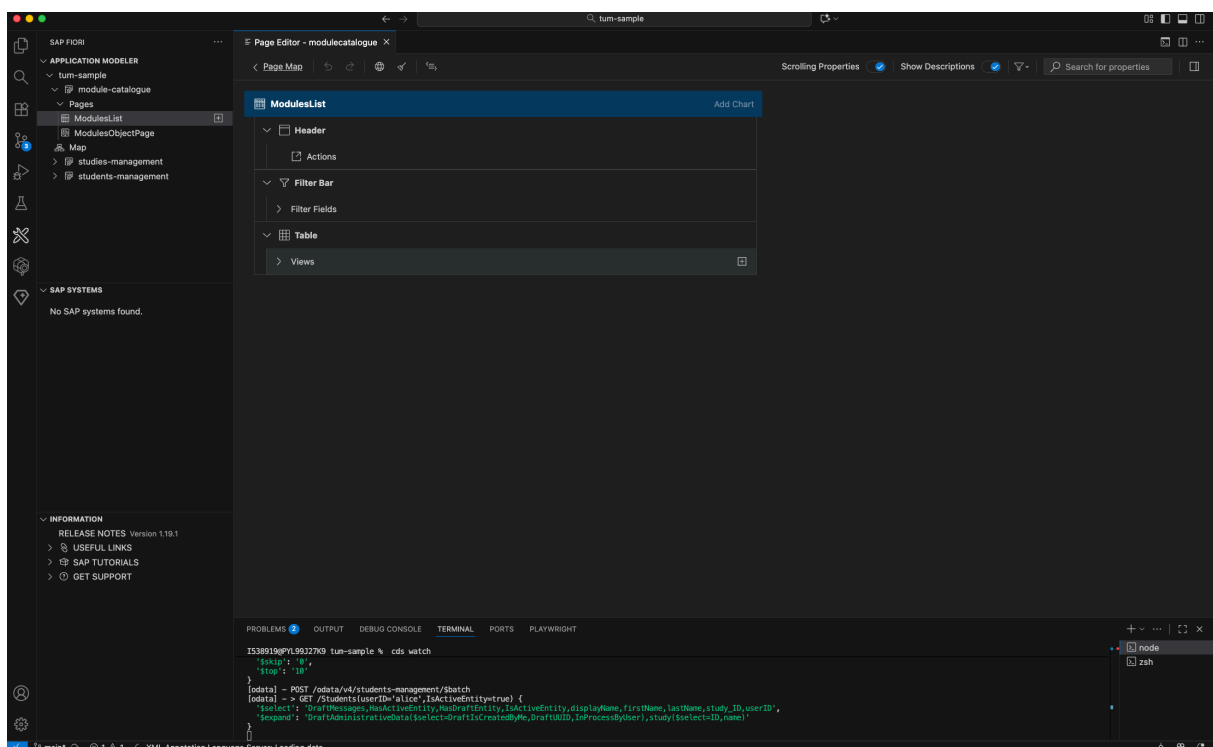
Sandbox Launchpad



4.12. Add Actions to UI

After the creation of the basic UI application, add Buttons to call the assign and unassign actions you created earlier. Again, use the Fiori Tools to open the Page Map of the catalog application or execute the Command **Fiori: Show Page Map** and select the catalog app.

Edit the Object Page:



- Under **Header** → **Actions** → **+** on the right side → **Add Actions** → Select the assign action

If you now restart, go to the object page and click assign, you should get an error that an implementation is missing. We will add that later.

4.13. Debug Your Application

You can start your application in debug mode by using a JavaScript Debug Terminal in VSCode and executing `cds watch` there.

4.14. Adding Authorization and Authentication

You now want to restrict the catalog service so that only Students can see it. Read about auth in CAP [here](#).

CAP offers pre-defined mock users. We now want to overwrite one of them to add the role Student to the mock user alice. You can do it like this in your `package.json`:

```
"cds": {
  "requires": {
    "[development]": {
      "auth": {
        "kind": "mocked",
        "users": {
          "alice": {
            "password": "1234",
            "roles": [
              "Student"
            ]
          }
        }
      }
    }
  }
}
```

4.15. Adding Business Logic

Run the following commands to add the TypeScript configuration:

```
cds add typescript
npm install
```

Add a `/srv/catalog-service.ts` file. CAP will automatically consider js/ts files with the same name as the service file.

```
import cds from '@sap/cds';
import { Module } from '#cds-models/CatalogService'

export class CatalogService extends cds.ApplicationService {
  init() {
    this.on(Module.actions.assign, async req => {
      console.log("Assign called", req.user)
    })
  }
}
```

Implement the assign action that assigns the user to the Module. In case an assignment already exists, do not add one but return an error.

- You can leverage `req.subject` as a shortcut for the information which entity is targeted by the bound action
- You can use `req.user` to gain information about the logged in user
- `req.params` contains the URL parameters like the ID of the bound entity
- Use CAPs SQL like query API to do the necessary statements
- Use `req.info` to let the student know that you assigned them or `req.error` to display the error
- Make use of the debugger to inspect the incoming requests or results from database access

Congratulations! You just created your first CAP-Application.

5. Optional: Advanced UI Customization

This section covers advanced UI enhancements to make your application more polished and user-friendly.

5.1. Example UI Screenshots

The following screenshots show what your finished applications could look like:

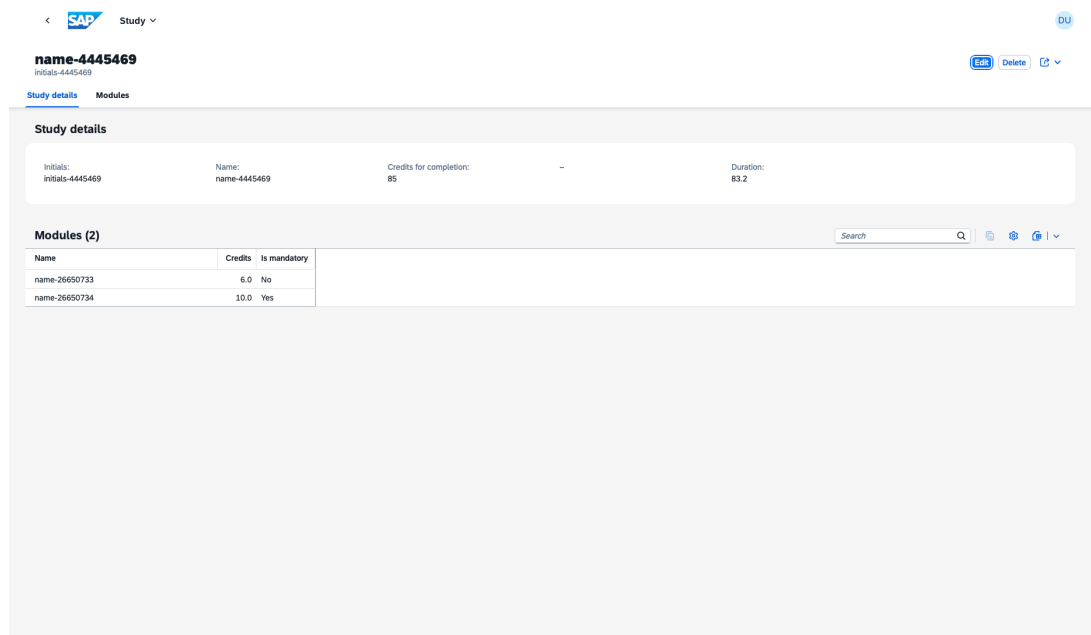
Students App:

User ID	First name	Last name	Study
☐ Alice Wonderland alice	Alice	Wonderland	name-4445469 >
☐ Bob the builder bob	Bob	the builder	name-4445469 >
☐ firstName-6519555 lastName-6519555 Students-6519555	firstName-6519555	lastName-6519555	name-4445469 >
☐ firstName-6519556 lastName-6519556 Students-6519556	firstName-6519556	lastName-6519556	name-4445469 >
☐ firstName-6519557 lastName-6519557 Students-6519557	firstName-6519557	lastName-6519557	name-4445469 >
☐ firstName-6519558 lastName-6519558 Students-6519558	firstName-6519558	lastName-6519558	name-4445469 >
☐ firstName-6519559 lastName-6519559 Students-6519559	firstName-6519559	lastName-6519559	name-4445469 >
☐ firstName-6519560 lastName-6519560 Students-6519560	firstName-6519560	lastName-6519560	name-4445469 >
☐ firstName-6519561 lastName-6519561 Students-6519561	firstName-6519561	lastName-6519561	name-4445469 >
☐ firstName-6519562 lastName-6519562 Students-6519562	firstName-6519562	lastName-6519562	name-4445469 >
☐ firstName-6519563 lastName-6519563 Students-6519563	firstName-6519563	lastName-6519563	name-4445469 >

Studies App:

Initials	Name	Credits for comple...
☐ Initials-4445370	name-4445370	54 >
☐ Initials-4445371	name-4445371	16 >
☐ Initials-4445372	name-4445372	12 >
☐ Initials-4445373	name-4445373	60 >
☐ Initials-4445374	name-4445374	77 >
☐ Initials-4445375	name-4445375	57 >
☐ Initials-4445376	name-4445376	31 >
☐ Initials-4445377	name-4445377	48 >
☐ Initials-4445378	name-4445378	0 >
☐ Initials-4445379	name-4445379	99 >
☐ Initials-4445380	name-4445380	79 >
☐ Initials-4445381	name-4445381	14 >
☐ Initials-4445382	name-4445382	24 >
☐ Initials-4445383	name-4445383	93 >
☐ Initials-4445384	name-4445384	36 >
☐ Initials-4445385	name-4445385	15 >
☐ Initials-4445386	name-4445386	74 >
☐ Initials-4445387	name-4445387	75 >

Module Catalog App:



5.2. Refresh UI with SideEffects

Use `@Common.SideEffects` to refresh parts of the UI when actions are executed. This ensures the UI reflects the latest data after an action like “assign” or “unassign” is pressed.

An example for having actions as a table column is given in the [Fiori Elements Showcase](#).

5.3. Multiple Tabs Configuration

Having multiple tabs can be configured via the [manifest.json](#) file of your application. This allows you to organize different content sections in a tabbed interface.

5.4. Row Highlighting with Criticality

Highlighting a row with color can be done via the so-called **Criticality** of the row ([Sample](#)). Instead of a property reference, you can use an expression:

```
isUserAssigned
```

Note: [Criticality Values](#): 3 = Positive, 0 = Neutral

5.5. Conditional Action Buttons

Annotating `@Core.OperationAvailable` to an action allows you to conditionally enable or disable the action button in the UI. You can specify a property from the bound entity or an expression.

```
@Capabilities.Deletable : false
@Capabilities.Insertable : false
entity Modules as projection on persistence.Modules {
    *,
    (exists assignments[student.userID = $user.id] ? true : false) as
    isUserAssigned: Boolean @(Ui.Hidden),
} actions {
    @(
        Core.OperationAvailable : (not $self.isUserAssigned),
        Common.SideEffects : {
            TargetProperties : ['in/isUserAssigned']
            TargetEntities : ['/moduleCatalog.EntityContainer/ModuleAssignments']
        }
    )
    action assign();
}
```

6. Optional: Deployment to Cloud Foundry

This section covers deploying your CAP application to SAP BTP Cloud Foundry.

6.1. Add Production Configuration

Run the following commands to add the necessary configuration for production deployment:

```
cds add xsuaa,hana --for production
```

This adds the configuration for:

- **XSUAA:** Authentication & Authorization service
- **HANA:** SAP HANA Cloud database

```
cds add mta,workzone --for production
```

This adds the configuration for:

- **MTA:** Multi-Target Application deployment descriptor
- **Workzone:** Central entry point for SAP solutions (Launchpad)

6.2. Fix ui5-deploy.yaml Configuration

Important: Due to a current bug, you need to manually add `relativePaths: true` to every generated `ui5-deploy.yaml` file, and add the `ui5-tooling-transpile-task`:

```

app > students-management > ! ui5-deploy.yaml
1  # yaml-language-server: $schema=https://sap.github.io/ui5-tooling/schema/ui5.yaml.json
2  specVersion: '3.1'
3  metadata:
4    name: students-management
5    type: application
6  resources:
7    configuration:
8      propertiesFileSourceEncoding: UTF-8
9  builder:
10   resources:
11     excludes:
12       - "/test/**"
13       - "/localService/**"
14   customTasks:
15     - name: ui5-task-zipper
16       afterTask: generateVersionInfo
17       configuration:
18         archiveName: students-management
19         relativePaths: true
20         additionalFiles:
21           - xs-app.json
22     - name: ui5-tooling-transpile-task
23       afterTask: replaceVersion
24       configuration:
25         debug: true
26         removeConsoleStatements: true
27         transformAsyncToPromise: true
28         transformTypeScript: true

```

6.3. Cloud Foundry Login

TODO: Update to login to their instance based on what is available.

6.4. Build and Deploy

Run the following commands to install dependencies and deploy your app to Cloud Foundry:

```

npm install
cds up

```

The `cds up` command bundles the build command from `mbt` with the `cf deploy` command for deployment.

6.5. Access Deployed Apps

To list your deployed HTML5 applications, check your `mta.yaml` for a service name ending with `-destination`. Copy that name and run:

```

cf html5-list --di <destination-name> -u

```

This will list the three HTML5 archives of your apps. In the URLs shown, replace `cpp` with `launchpad` to access the applications.

Note: The Module catalogue app might not open if role restrictions are defined, depending on your landscape configuration.

6.6. Cleanup / Undeploy

To undeploy your application and clean up resources, run:

```
cf undeploy <ID from mta.yaml> --delete-services
```

Replace <ID from mta.yaml> with the actual ID specified in your mta.yaml file.